PRD: MediVault (Role-Based Medical Records Portal)

Product requirements document, v0.1. Learning project, fictional data only. Not a certified or HIPAA-compliant system.

1. Header

Field	Value
Product	MediVault – Role-Based Medical Records Portal with Patient Access Delegation and Audit Log
ID/Track	PST_PD_xxx (assigned by mentor) / Product Development
Batch / Level	2026 Stretch
Duration	4 weeks (one-month OJT)
Student/Team	Student fills (individual or 2-student team)
PRD version	v0.1

2. Overview

A one-month stretch-track OJT project to build MediVault, a small medical records portal demo for Doctors, Patients and Medical Records Officers. A front-end application built with HTML, CSS, JavaScript and Bootstrap 5 shows each role only the records and actions it is allowed to use. Patients can delegate access to a doctor for chosen record categories and a limited time, and can revoke it at any moment. Every access attempt, allowed or denied, writes an entry to an append-only audit log. Access rules sit in separate, testable JavaScript modules so that automated tests can prove that expired or revoked delegations stop working and that no unauthorized access succeeds. The stretch scope adds HIPAA-style controls (session timeout, minimum-necessary views, break-glass access, tamper-evident log) and a React + Vite rebuild of one module as self-study. This is a learning project. It makes no HIPAA, GDPR or other compliance claim.

- Industry: Health information systems and access governance
- Theme: Role-based access, patient-controlled delegation and auditability

- Product promise: Medical records are visible only to the people the patient and the role rules allow, only for the time allowed, and every access is recorded.
- Primary objective: Learn front-end architecture for a sensitive, role-based application: role guards, state handling, safe rendering, expiry logic and audit logging, with evidence from automated tests.
- Guiding rule: Don't overbuild. Finish the role model, delegation with expiry, the audit log and the adversarial tests first. Start HIPAA-style controls and the React + Vite self-study only after the core tests pass.

3. Problem statement

Patient information is often shared through paper files, email attachments and shared folders. Patients cannot see who looked at their records, access given to a doctor is rarely taken back, and staff see more than they need. MediVault provides a small portal where each role has a clear, minimal view, the patient decides who may see which records and for how long, and an audit log answers 'who accessed what, and when'.

Student fills:

- Primary problem statement and the specific workflow being improved (patient delegation, clinical record access, or record administration and audit).
- Three research questions: (1) How are patient records currently shared between doctors, patients and records staff, and what goes wrong? (2) Which actions must be audited to answer 'who accessed which record, and when'? (3) What must each role see, for example can a Medical Records Officer manage records without reading clinical content?
- Hypothesis: a single access check that evaluates role, care assignment, delegation scope and expiry and writes the audit entry in the same step, combined with minimum-necessary screens, leaves no way to open a record without leaving a log entry.
- The decision the product supports: whether a user may open a record, whether a delegation is still valid, and whether an access was legitimate.

4. Personas

Persona	Primary need	Expected benefit
Doctor	Read and add clinical entries for patients who have granted access; see which patients have shared records	Fast, legitimate access to the records needed for care, with a clear reason for every denial

Persona	Primary need	Expected benefit
Patient	View own records; grant, limit and revoke doctor access; see who accessed the records	Control over personal data and a visible access history
Medical Records Officer (MRO)	Register patients, assign doctors to patients, manage record index and correction requests, review the audit log	Administrative control and oversight without reading clinical content

Student task: Create persona cards describing each user's responsibilities, goals, common frustrations, permissions and preferred information. Define what each role can and cannot do. For example: a Doctor can open a record only if the patient has an active delegation covering that category and the doctor is assigned to the patient; a Patient can read only their own records and manage only their own delegations; an MRO can manage administrative data and the audit log but cannot open clinical content; nobody can edit or delete audit entries.

5. Key user journeys

Journey 1: Patient delegates access to a doctor

1. The patient signs in with the Patient role (demonstration login) and opens Access Delegation.
2. The patient chooses a doctor, the record categories to share (for example Lab results, Prescriptions) and an expiry (1 hour to 30 days).
3. The system shows a summary of what will be shared and until when, and asks for confirmation.
4. The delegation is saved as Active and an audit entry is written.

Journey 2: Doctor opens a patient record

1. The doctor signs in and sees the patient list with the delegation status for each patient.
2. The doctor opens a patient with an active delegation; only the shared categories are shown.
3. The doctor tries a category that was not shared, or a patient with an expired or revoked delegation; the system refuses with a generic message and logs the attempt as Denied.
4. The patient later sees both the allowed and the denied attempts in their access history.

Journey 3: Doctor adds a clinical entry

1. The doctor selects a patient with an active delegation that covers the category.
2. The doctor adds a visit note, prescription or lab result with inline validation.
3. The entry is saved with author and time, shown in the patient's record and logged without its content.

Journey 4: Medical Records Officer administers and audits

1. The MRO signs in, registers a patient and assigns a doctor to that patient.
2. The MRO opens the record index and sees titles, categories and dates, never clinical content.
3. The MRO filters the audit log by patient, actor, action, outcome and date, with denied and revoked entries highlighted.
4. If the MRO tries to open clinical content, the system refuses and logs the attempt.

Journey 5: Patient revokes access and reviews history

1. The patient opens the delegation list and revokes one delegation after a confirmation.
2. The doctor's next attempt is refused immediately and logged as Denied.
3. The patient opens Access History and filters it by doctor, action and date.

Journey 6: Break-glass emergency access (stretch scope)

1. A doctor without delegation selects Emergency access for an assigned patient and must enter a reason.
2. The system grants a short, time-limited, read-only view, marks the entry as Break-glass and logs it.
3. The patient sees the break-glass entry highlighted in their access history, and the MRO sees it flagged for review.

6. Scope

Must have (MVP – graded)

- HTML, CSS, JavaScript (ES6) and Bootstrap 5 application with simulated sign-in for three roles (Doctor, Patient, MRO) using fictional demo accounts.
- Role-based navigation and screens; every action goes through one `authorize(user, action, resource, now)` function that is separate from the UI.
- Patient records (fictional): categories Visit notes, Lab results, Prescriptions, Allergies and Imaging reports; list, view and add by authorized roles.

- Patient access delegation: grant to a doctor with category scope and expiry, list, revoke; Active, Expired and Revoked states derived from time and records.
- Care assignment: the MRO assigns doctors to patients; a doctor needs both an assignment and a valid delegation to open clinical content.
- Minimum-necessary views: the MRO sees administrative data and the record index only; lists show metadata, content opens only through an explicit, logged open action.
- Audit log: append-only entries for every sign-in, record open, record add, delegation change, denied attempt and automatic expiry, with timestamp, actor, role, patient, action, outcome and reason; entries never contain clinical content.
- Patient access history view: the patient sees who accessed their records and when.
- Audit log screen with filters (patient, actor, action, outcome, date) and highlighted denied entries.
- Input validation: required fields, name and text length limits, allowed characters, delegation expiry range, escaping of all rendered text.
- Sign-in protection: demo PIN stored as a salted hash, temporary lockout after repeated failures.
- Sample data (fictional patients, doctors and records) with a reset option.
- Automated tests (Node): unit tests per module, a delegation-expiry test with 50 or more checks using an injectable clock, and an adversarial test suite.
- README with threat model and known limitations.

Stretch scope (only after the core tests pass): HIPAA-style controls

- Session timeout with a warning and automatic sign-out after inactivity; screens clear on sign-out.
- Break-glass emergency access with mandatory reason, short time limit and flagged log entry.
- Minimum-necessary field masking (for example patient identifiers partly hidden in lists).
- Hash-chained audit entries with a verify-integrity check to detect tampering.
- Patient-facing report of disclosures (who saw what, within a date range) as printable view.
- Record correction requests by the patient, handled by the MRO.
- Stated clearly in the README as HIPAA-style practice exercises, not compliance.

Self-study track: React + Vite

- Rebuild one module (the Patient Access Delegation screens or the Audit Log table) in React + Vite, reusing the same logic modules.

- Components and hooks for state, a simple role-aware router, and a short comparison note: plain JavaScript vs React.

Good to have (week 3 onward, only if the core and tests already pass)

- Encryption of stored records with the Web Crypto API (demonstration only).
- Export of the audit log to CSV and a Chart.js access chart.
- Dark theme toggle.
- Print-friendly patient summary.

Out of scope

- Real medical data, real patient identities or any real health information.
- HIPAA, GDPR or any other compliance certification or claim.
- Backend server, database, real authentication, SSO or OAuth.
- Integration with real hospital systems or standards (HL7, FHIR) and e-prescribing.
- Production-grade cryptography and key management.
- Public deployment or app store release.

7. User stories

ID	User story	Acceptance criteria
US-01	As a patient, I want to view my own records so that I know what is stored about me.	Only the signed-in patient's records are listed; other patients' data never loads.
US-02	As a patient, I want to grant a doctor access to selected categories for a limited time so that I control sharing.	Doctor, categories and expiry are validated; a confirmation summary is shown; the delegation is saved as Active and logged.
US-03	As a patient, I want to revoke a delegation so that I can react to a concern.	Revoke needs confirmation; status becomes Revoked immediately, is final and is logged.
US-04	As a patient, I want to see who accessed my records so that I can spot misuse.	Access history lists allowed and denied attempts with actor, action, time and outcome, without clinical content.
US-05	As a doctor, I want to open records of patients who shared them so that I can provide care.	Content opens only with assignment, active delegation and matching category; the open is logged.

ID	User story	Acceptance criteria
US-06	As a doctor, I want a clear refusal when access is not allowed so that I know what to do.	Missing, expired, revoked or out-of-scope access is refused with a generic message and logged as Denied.
US-07	As a doctor, I want to add a clinical entry so that the record stays current.	Entry is validated, saved with author and time, and logged without content.
US-08	As an MRO, I want to register patients and assign doctors so that care teams are set up.	Registration and assignment are validated, saved and logged; no clinical content is shown.
US-09	As an MRO, I want to see a record index without clinical content so that I can manage records safely.	Index shows title, category and date only; opening content is refused and logged.
US-10	As an MRO, I want to filter the audit log so that I can investigate.	Filters by patient, actor, action, outcome and date work together; no clinical content is shown.
US-11	As any user, I want repeated wrong sign-in attempts blocked so that guessing fails.	After 5 failures sign-in is locked for 60 seconds with the remaining time shown; attempts are logged.
US-12	As a patient, I want expired delegations to stop working automatically so that access does not linger.	At the exact expiry time access fails, status becomes Expired and an automatic-expiry log entry exists.
US-13	As a doctor in an emergency (stretch), I want break-glass access so that care is not delayed.	A reason is required; access is read-only and time-limited; the entry is flagged for patient and MRO.
US-14	As any user, I want clear validation messages and a layout that works on my phone so that I can use the portal anywhere.	Errors appear beside fields and describe the fix; flows work at about 360 px width without horizontal scrolling.

8. Screen inventory

Screen	Key elements	Primary action
Sign-in	Role selector, demo account, PIN field, lockout message	Sign in

Screen	Key elements	Primary action
Dashboard	Role-specific summary cards, shortcuts, current role	Open a task
My Records (Patient)	Category tabs, record list, record viewer	Open a record
Access Delegation (Patient)	Grant form (doctor, categories, expiry), delegation list with status badges	Grant or revoke access
Access History (Patient)	Table of who accessed what, filters, denied entries highlighted	Review activity
Patient List (Doctor)	Assigned patients, delegation status, search	Open a patient
Patient Record (Doctor)	Shared categories, entries, add-entry button	Read or add an entry
Entry Form (Doctor)	Category, title, content, inline validation	Save an entry
Patient Registry (MRO)	Register form, patient list, doctor assignment	Register or assign
Record Index (MRO)	Titles, categories, dates, no content	Manage the index
Audit Log	Log table, filters (patient, actor, action, outcome, date), highlighted denied and break-glass entries	Review activity
Emergency Access (Doctor, stretch)	Reason field, time limit notice, confirm	Request break-glass access
Settings / Reset	Reset sample data, current role, session timer	Reset demo data

9. Component breakdown

Component	Responsibility	Key inputs / events
store.js	Load and save app data in localStorage with a schema version, safe parse and reset to sample data	Data, storage key

Component	Responsibility	Key inputs / events
auth.js	Demo sign-in, PIN hashing with salt, lockout, session and timeout handling	Role, PIN, now
access.js	One `authorize(user, action, resource, now)` function; role, assignment, delegation scope and expiry checks	User, action, resource
delegation.js	Grant, revoke, derive status, expire delegations; injectable clock	Patient, doctor, categories, expiry, now
records.js	List, open, add records; return metadata or content depending on the checked permission	Patient id, category, actor
audit.js	Append-only audit log with sequence numbers; filtering; guarantees no clinical content is written	Actor, action, resource, outcome
validators.js	Validate names, text lengths, categories, expiry range and ids	Input values
render.js	Escape and render text safely (textContent, no unsafe HTML)	Strings, data
app.js (UI)	Routing between screens, role-based navigation, event handling	Events, state
RecordViewer (UI)	Display one record and clear it from the page on close or sign-out	Record, close event
DelegationList (UI)	Render delegations with status badges and revoke controls	Delegations
AuditLogTable (UI)	Render and filter log entries with outcome badges	Log entries, filters
React module (self-study)	One screen rebuilt with components, hooks and the shared logic modules	Props, state

Student task: Define the JSON schema, function signatures, error codes (for example ERR_FORBIDDEN, ERR_NOT_FOUND_OR_DENIED, ERR_DELEGATION_EXPIRED, ERR_SCOPE, ERR_LOCKED, ERR_NOT_ASSIGNED, ERR_INVALID_INPUT), log fields and a short threat model, and write automated tests for the logic modules (store, auth, access, delegation, records, audit, validators).

10. State and data

Core data entities

Entity	Important fields	Purpose
User (Demo)	userId, name, role, pinHash, salt, failedAttempts, lockedUntil	Demonstration account for Doctor, Patient or MRO
Patient	patientId, userId, name, dateOfBirth, registeredAt	A fictional patient profile
CareAssignment	assignmentId, patientId, doctorId, assignedBy, assignedAt, status	Links a doctor to a patient
RecordEntry	recordId, patientId, category, title, content, authorId, createdAt, sensitivity	One clinical entry
Delegation	delegationId, patientId, doctorId, categories, grantedAt, expiresAt, status, revokedAt	Patient-granted, time-limited access
AuditLogEntry	logId, timestamp, actor, role, action, patientId, resourceName, outcome, reason, flag, prevHash (stretch)	Append-only audit record without clinical content
CorrectionRequest (stretch)	requestId, patientId, recordId, reason, status	Patient request to correct a record

A delegation's effective status is derived: it is Active only while the current time is before expiresAt and it has not been revoked. Remaining time, masked display, log filters and counts are calculated from records, not stored.

State management

- The app keeps an in-memory copy loaded from localStorage and saves it after every change; storage can be cleared or full, so every read and write is wrapped in error handling.

- UI state is kept in JavaScript objects. Opened clinical content lives only in a variable that is cleared on close, on sign-out and on session timeout. It is never written to URLs, the console or the audit log.
- Time handling: one clock function is used everywhere so that tests can replace it and control time exactly.
- Derived values: delegation status, remaining time, masked display and log summaries are calculated, not stored.

Workflow rules

1. Text fields are validated for length and allowed characters; all rendered text is escaped; names 1 to 80 characters, titles 1 to 100, entry content 1 to 2000, reasons up to 200.
2. Lists and search results show metadata only; clinical content is returned only by an explicit open action that passes `authorize`.
3. Every access attempt, successful or denied, writes exactly one audit entry in the same operation as the decision. If the entry cannot be written, access is denied (fail closed).
4. Authorization: a Patient may read only their own records and manage only their own delegations; a Doctor may open content only when assigned to the patient and holding an active delegation for that category; an MRO may manage administrative data and read the audit log but never clinical content; nobody may edit or delete audit entries.
5. Denied responses are generic and do not reveal whether a patient, record or delegation exists.
6. A delegation has an expiry between 1 hour and 30 days; it is valid only while now is before expiresAt and it has not been revoked.
7. At or after expiresAt access fails immediately (check on use) and a periodic check (at most every second while the app is open) sets status Expired and writes one automatic-expiry audit entry. An expired or revoked delegation can never become Active again; a new grant creates a new delegation.
8. Demo PINs are compared safely; after 5 failed sign-ins the account is locked for 60 seconds; failures are logged.
9. The audit log is append-only: no function or screen edits or deletes entries; entries never contain clinical content, PINs or session data.
10. Stored data has a schema version; corrupt data is detected on load with a clear error and an offer to reset sample data.
11. Stretch: break-glass requires a reason, is read-only, lasts at most 15 minutes and is flagged; inactivity beyond 5 minutes signs the user out.

Known limitation to document: This is a front-end-only learning project, not a medical or compliance-grade system. Data in localStorage is readable by anyone with access to the browser, and front-end role checks can be bypassed by someone with developer tools; they demonstrate the model, they are not real security. A real system needs a server-side authorization layer, strong authentication, encryption, TLS and legal and regulatory review. HIPAA-style controls are practice exercises, not a compliance claim.

11. UI states

State	Expected behavior
Empty	Explain no records, delegations or patients exist and offer the next action
Masked	Show metadata and a fixed 'Content hidden' label until a record is opened
Revealed	Show the record content with a clear Close button; content clears on close, sign-out or timeout
Access Denied	Generic message that does not reveal whether the record exists; the attempt is logged
Delegation Active	Badge 'Active' with remaining time and the shared categories
Delegation Expired or Revoked	Badge 'Expired' or 'Revoked' with text and icon, not only color, and the reason
Locked Out	Explain sign-in is locked and show the remaining time
Session Timeout (stretch)	Warn before sign-out, then sign out and clear screens
Break-glass Active (stretch)	Persistent banner with remaining time and a read-only notice
Unauthorized Action	Explain the current role cannot perform the action and leave data unchanged
Success	Toast confirming saved, granted, revoked or registered, never showing clinical content
Loading	Show a brief indicator while sample data loads
Invalid Form	Highlight fields and show actionable messages

State	Expected behavior
Storage Error	Explain data could not be saved or read and offer to reset sample data

12. Responsiveness

Device	Layout	Expected behavior
Phone	Single column, collapsible navbar, tables become cards	Review records, delegations and history with thumb-friendly controls
Tablet	Two-column layout for lists and detail	Manage patients and delegations comfortably
Laptop	Navbar, side filters, tables for records and audit log	Complete every role flow

- Use the Bootstrap 5 grid, utilities and components (navbar, cards, modal, toast, forms, badges, tables, tabs).
- Wide tables scroll inside a responsive wrapper or become cards on small screens.
- Keyboard navigation, visible focus states and touch targets of at least 44 px.
- Test at about 360 px, 768 px and 1280 px.

13. Visual design, accessibility and UX copy

- Visual style: Calm, clinical and uncluttered; neutral base with one accent; sensitive content looks 'closed' until deliberately opened.
- Color: Semantic colors for Active, Expired, Revoked, Allowed, Denied and Break-glass that are also distinguished by text or icon.
- Accessibility: Semantic HTML, associated labels, accessible modals and toasts, keyboard-operable open and close, countdowns announced politely to screen readers, clear role and status text.
- Contrast: Target WCAG 2.2 AA.
- Confirmation: Always confirm revoke and delegation grant; show the categories and expiry time before granting.
- UX copy: Short and specific, for example 'Access denied.' (never 'No such record') and 'Shared until 14 Jun, 18:00'.

- Privacy and safety: Use only fictional patients and records; show a visible 'Demo data only' notice. Never show clinical content in toasts, URLs, console output or audit entries.

14. Tech stack

- Front end: HTML5, CSS3, JavaScript (ES6 modules) and Bootstrap 5, served as static files.
- Persistence: browser localStorage with a schema version and reset; no backend or database.
- Self-study: React + Vite for one module, with components, hooks and the shared logic modules.
- Testing: Node test runner (or Jest) for the logic modules with an injectable clock; an adversarial test script; a manual checklist for the screens, keyboard use and responsive widths.
- Local run: open through a simple static server (for example Vite dev server or `npx serve`).
- Version control: Git and GitHub with clean commits; never commit real data.
- Deployment: reproducible local demo with a screen recording; optional static hosting with fictional data only.
- Optional self-study: Web Crypto encryption, hash-chained audit log, Chart.js access chart.

15. Learning goals

- Course covered: Frontend Development Basics.
- Write modular JavaScript (ES6): functions, modules, objects, arrays, exceptions and unit tests.
- Build responsive, accessible interfaces with Bootstrap 5 and handle forms with clear validation.
- Handle time correctly: expiry, a replaceable clock and periodic checks.
- Apply secure front-end basics: escaping output, input validation, least privilege, fail-closed design and safe logging.
- Model a role-based workflow and keep access-control logic separate from the UI.
- Manage application state and browser storage safely.
- Stretch (self-study): React components, props, state and hooks with Vite; tamper-evident logs.
- Use Git and GitHub for version control and collaboration.

16. Milestones

Phase	Plan	Expected output
Week 1	Finalize requirements, threat model, personas, journeys and wireframes; Git repository; JSON schema; store, validators and records modules with unit tests	Repo, threat model, wireframes, working data modules with tests
Week 2	Sign-in and roles; `authorize` function; delegation module; audit log; Bootstrap screens for patient, doctor and MRO	Working role flows with masked lists, open and audit log
Week 3	Expiry and automatic revocation; 50-check expiry test; adversarial test suite; responsive and accessibility pass. Start stretch only if core tests pass	Passing expiry and adversarial tests
Week 4	HIPAA-style stretch controls (timeout, break-glass, hash chain) and React + Vite module as time allows; README; demo	MVP release, test results, demo, README

Priority: Deliver working role flows with passing expiry and adversarial tests before any stretch or React feature.

17. Evaluation

Proposed measurable checks (confirm with mentor):

- Delegation expiry test: at least 50 checks with varied expiries (including the shortest allowed, identical expiry times and many created together) using an injectable clock. One tick before expiresAt access is allowed; at expiresAt it is refused; every expired delegation has exactly one automatic-expiry log entry; zero failures allowed.
- Adversarial test covers at least: Doctor without assignment, without delegation, with expired or revoked delegation and with out-of-scope category; Patient opening another patient's record; MRO opening clinical content; Doctor editing or deleting audit entries; tampered localStorage data; tampered role or user id in storage; script text in names and notes (escaping check); oversized and special-character inputs; repeated wrong PINs; replaying a revoked delegation; revoke and open at the same

time; damaged storage data; and checking that no clinical content or PIN appears in any audit entry or error message. Expected result: zero unauthorized successes and every denied attempt logged.

- Role flows: a new user can complete the delegation, doctor-access, MRO-administration and audit-review tasks without help in a short walkthrough with two or three peers, with results noted.
- Usable screens: no horizontal scrolling at about 360 px, visible focus states and labelled form fields on every screen.
- Stretch checks: inactivity timeout signs out and clears content; break-glass entries are flagged; the hash-chain check detects a modified entry.

Suggested evaluation rubric

Evaluation area	Weight
Access control, delegation and expiry logic (including tests)	25%
Role flows and audit log (doctor, patient, MRO)	15%
Web UI quality, responsiveness and accessibility	15%
Code structure, JavaScript quality and data handling	15%
Testing and validation (including adversarial tests)	10%
Stretch scope (HIPAA-style controls, React + Vite self-study)	10%
Documentation and final demonstration	10%
Total	**100%**

Acceptance criteria

1. Patients can view their own records and grant, list and revoke delegations.
2. Doctors can open only records of assigned patients covered by an active delegation and category.
3. The MRO can register patients, assign doctors and review the audit log, and never sees clinical content.
4. Clinical content never appears in lists, audit entries or error messages.
5. Every access attempt, allowed or denied, is written to the audit log, and nobody can edit or delete entries.

6. Delegations expire automatically at the exact expiry time across 50 or more test checks.
7. The adversarial test shows zero unauthorized access.
8. Sign-in lockout works after repeated failures.
9. The interface is usable on phone and laptop widths with clear validation and feedback.
10. Stretch: at least two HIPAA-style controls and one React + Vite module are working and documented as practice, not compliance.
11. The README explains setup, roles, sample data, the threat model, test results and known limitations.

Expected output: A working MediVault front-end portal with role-based screens, patient access delegation with automatic expiry, an audit log and patient access history, automated expiry and adversarial test results, stretch HIPAA-style controls and a React + Vite module as time allows, a GitHub repository, README and a reproducible demo.

18. Risks and assumptions

Risk / assumption	Potential impact	Mitigation
Project mistaken for a compliant medical system	False sense of security and compliance	State clearly in the README, footer and demo that it is a learning project with no compliance claim
Clinical content leaks into logs, errors or URLs	Defeats the purpose of the project	Never log content; add tests that scan logs and error messages for content
Expiry drift or inconsistent time handling	Access valid after expiry	Use one clock function, check expiry on every use and also check periodically; test boundaries
Front-end role checks mistaken for security	Misleading assumption	State clearly that checks are for demonstration; keep the guard inside the logic modules, not the UI
localStorage readable, cleared or full	Data exposure or loss	Fictional data only, reset option, clear error messages, document the limit

Risk / assumption	Potential impact	Mitigation
Unescaped text rendered as HTML	Script injection in the page	Render with textContent, validate input, add adversarial tests
Real personal or health data entered	Privacy risk	Fictional data only and a visible notice
Stretch work crowds out core work	Core tests unfinished	Start stretch only after expiry and adversarial tests pass
React + Vite self-study takes too long	Incomplete module	Limit to one module; keep the plain JavaScript version as the graded one
Scope grows beyond a one-month OJT	Incomplete core flow	Finish the core flow and tests first
Accessibility ignored until the end	Poor usability	Check labels, focus and contrast every week

19. Deliverables

- Product requirements document, threat model and user journey diagrams.
- Wireframes for the web screens of all three roles.
- GitHub repository with clean commits and a documented folder structure (no real data committed).
- HTML, CSS, JavaScript and Bootstrap 5 application with role-based screens, masked metadata and audit log.
- Role-based access for Doctor, Patient and Medical Records Officer.
- Patient access delegation with scope, expiry and automatic revocation.
- Sample dataset (fictional patients, doctors and records) with a reset option.
- Automated test suite and results: unit tests, 50-check expiry test and adversarial test.
- Stretch: HIPAA-style controls (session timeout, break-glass, hash-chained log) and a React + Vite module.
- README with setup instructions, architecture overview, threat model, screenshots, role instructions and known limitations.
- Reproducible local demo, plus a demo video or live presentation.

Student task: Document exact tool versions (Node.js, Vite, React, browser), the JSON schema, the access and expiry rules, the test cases and known limitations.

20. Sign-off

Role	Name	Date	Approved
Student 1	Emon Kalyan	01/11/2026	**Approved**
Student 2			
Mentor / Reviewer			